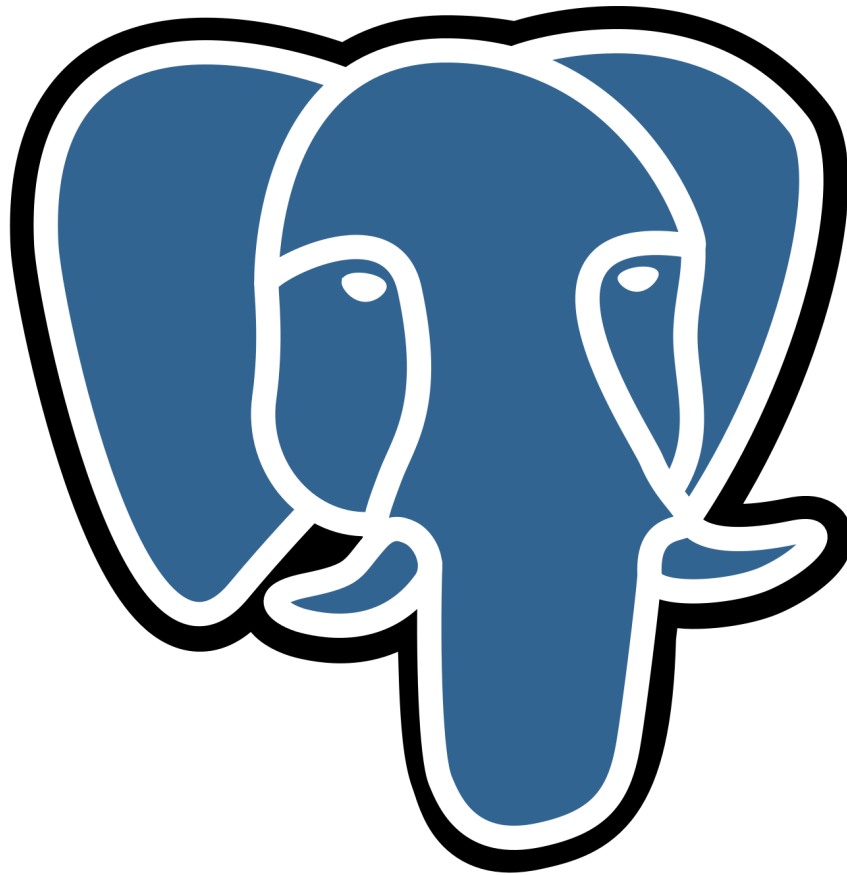


De PostgreSQL a la Web: datos relacionales en formato JSON



Nombres:

José Alejandro Díaz Delgado

Alejandro Tigreros Grajales

Nombre del módulo : Lenguajes de marcas

ÍNDICE

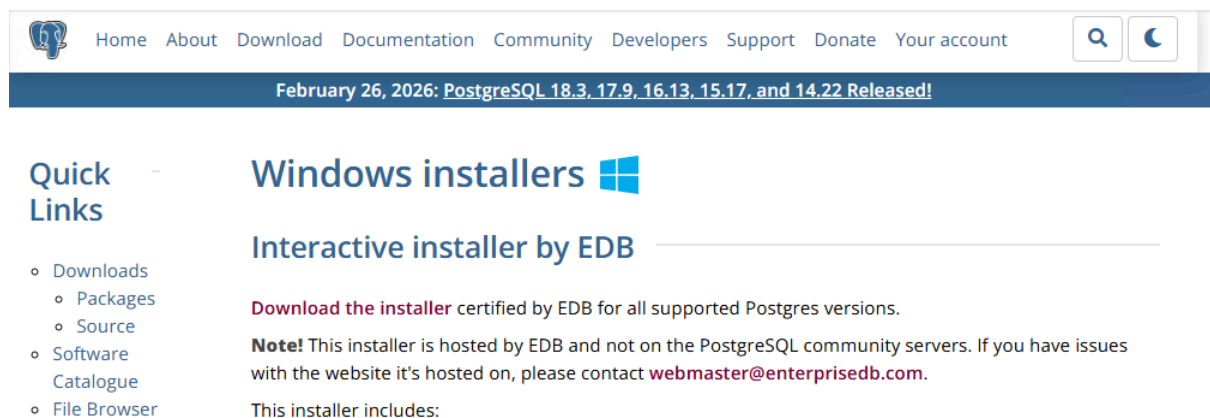
ÍNDICE.....	1
INTRODUCCIÓN.....	2
Instalación y Configuración de PostgreSQL.....	2
Diseño e implementación de la base de datos.....	6
Consultas sql y exportación a JSON.....	10
Aplicación web con JavaScript básico.....	16
CONCLUSIÓN.....	20
WEBGRAFÍA:.....	20

INTRODUCCIÓN

En este trabajo de datos relacionales entre PostgreSQL y la web nos centraremos sobre el tema de catálogo de videojuegos, utilizaremos

Instalación y Configuración de PostgreSQL

Primero descargamos en la máquina virtual Postgresql

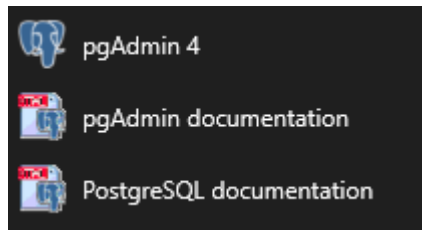


The screenshot shows the PostgreSQL website's 'Windows installers' page. The navigation bar includes links for Home, About, Download, Documentation, Community, Developers, Support, Donate, and Your account. A banner at the top of the main content area reads 'February 26, 2026: PostgreSQL 18.3, 17.9, 16.13, 15.17, and 14.22 Released!'. On the left, under 'Quick Links', there are links for Downloads, Packages, Source, Software Catalogue, and File Browser. The main heading is 'Windows installers' with a Windows logo. Below it, the section is titled 'Interactive installer by EDB'. The text states: 'Download the installer certified by EDB for all supported Postgres versions.' A note follows: 'Note! This installer is hosted by EDB and not on the PostgreSQL community servers. If you have issues with the website it's hosted on, please contact webmaster@enterprisedb.com.' It then says 'This installer includes:'.

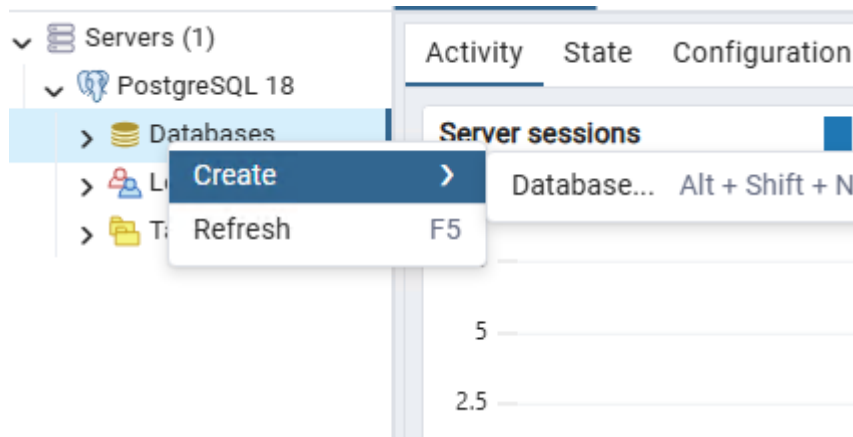
Ejecutamos el instalador y procedemos a la instalación



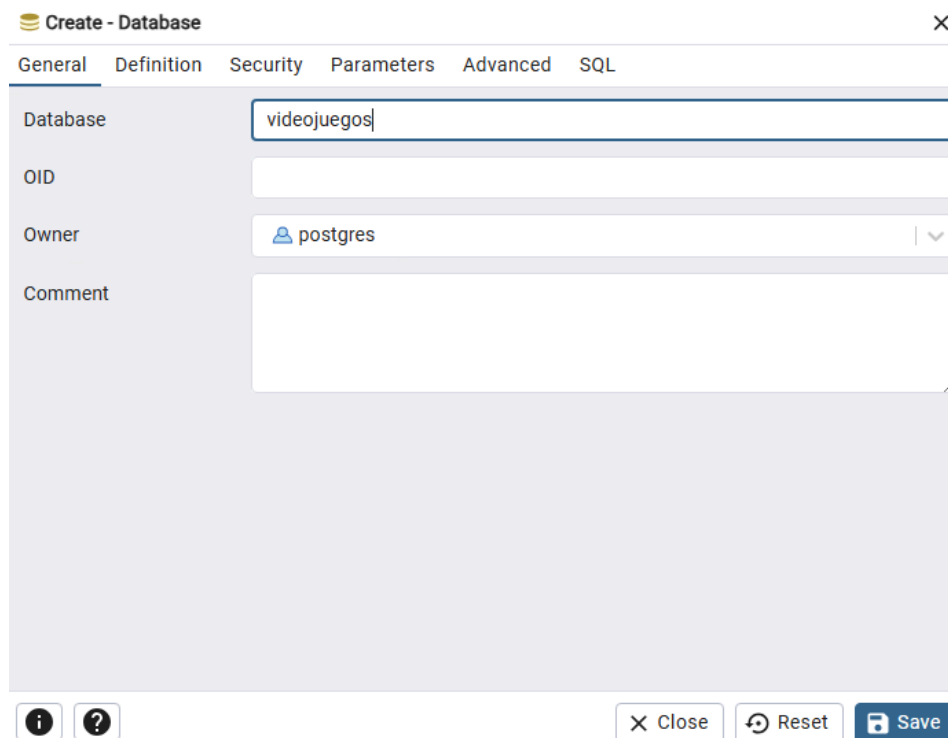
Al acabar con la instalación nos dirigimos a pgAdmin 4



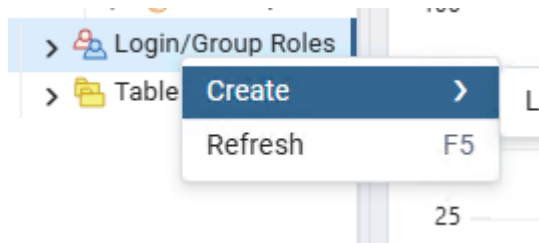
Ahora creamos una nueva base de datos



Le añadimos los datos necesarios



Ahora para tener un mejor manejo sin errores de nuestra base de datos vamos a crear un usuario



y le damos los permisos que sean necesarios



Luego de crearla, click derecho a la base de datos, propiedades y luego de eso security y dentro añadimos el usuario con los permisos

✕

General
Definition
Security
Parameters
Default Privileges
Advanced
SQL

Privileges

Grantee

🗑️

👤 jose

▼

Privileges

CTc

☒ ALL
☐ WITH GRANT OPT

☒ CREATE
☐ WITH GRANT OPT

☒ TEMPORARY
☐ WITH GRANT OPT

☒ CONNECT
☐ WITH GRANT OPT

Grantor

👤 postgres

Security labels

Provider

Security label

y con esto ya estaría configurada nuestra base de datos

Diseño e implementación de la base de datos

Para el diseño de la base de datos utilizaremos ‘Draw.io’, en él estableceremos la estructura que tendrá nuestra base de datos con 5 tablas que tengan conjunto.

```

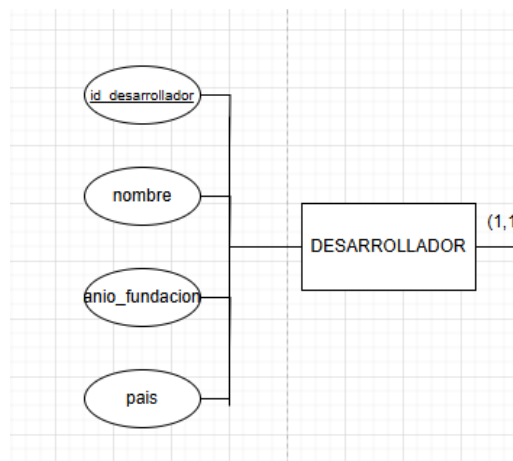
erDiagram
    DESARROLLADOR ||--o{ VIDEOJUEGO : "DESARROLLA"
    VIDEOJUEGO ||--o{ GENERO : "DESARROLLA"
    VIDEOJUEGO ||--o{ SAGA : "PERTENECE"
    VIDEOJUEGO ||--o{ PLATAFORMA : "DISPONE"
    SAGA ||--o{ VIDEOJUEGO : "PERTENECE"
    PLATAFORMA ||--o{ VIDEOJUEGO : "DISPONE"

    DESARROLLADOR {
        string id_desarrollador PK
        string nombre
        int año_fundación
        string país
    }
    VIDEOJUEGO {
        int año_lanzamiento
        string id_videojuego PK
        string título
        float precio
        int id_desarrollador FK
        int id_saga FK
    }
    GENERO {
        string id_genero PK
        string nombre
    }
    SAGA {
        string id_saga PK
        string nombre
        int año_inicio
    }
    PLATAFORMA {
        string id_plataforma PK
        string nombre
        string Fabricante
    }
  
```

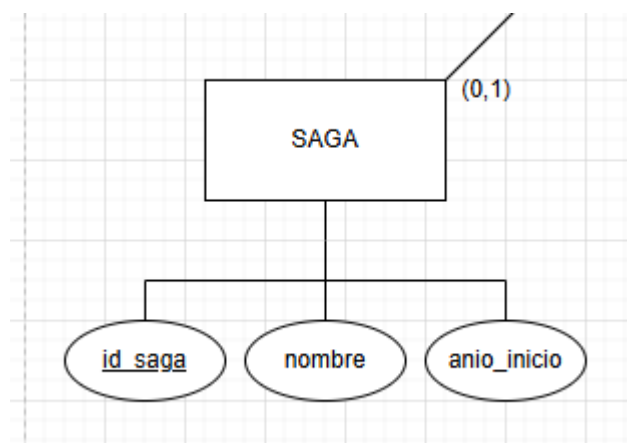
5

Las 5 tablas que aparecen son las que se muestran en la imagen, para empezar a explicar el diseño de esta es importante adentrarse en cada uno de los puntos importantes

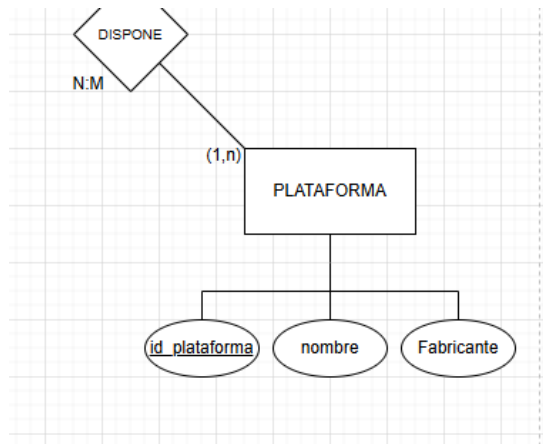
Dentro de nuestro catálogo de videojuegos es importante que en la base de datos exista un apartado de cada uno de los desarrolladores que han creado el videojuego, hemos pensado en que pueden ser varios desarrolladores que trabajen de forma simultánea como sucede con algunos títulos sin embargo, aquí existe un pequeño problema y es que quedaría ambiguo en el diagrama, y aunque trabajen varias compañías normalmente en el título escogeremos únicamente el que siempre queda como el desarrollador principal del título.



Esta tabla tiene 4 atributos tal y como se muestra en el diagrama.

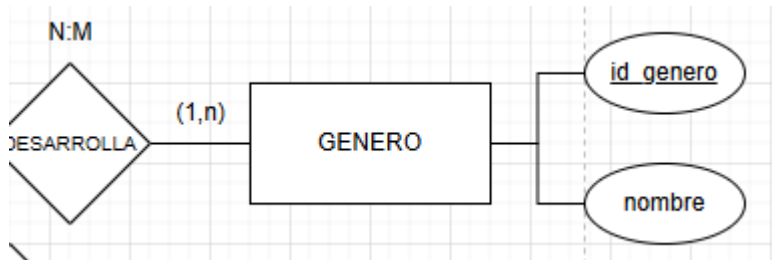


Por otro lado hemos añadido una tabla llamada saga, esta tabla es opcional ya que no todos los títulos deben pertenecer a una saga, esta tabla tiene 3 atributos, uno de ellos



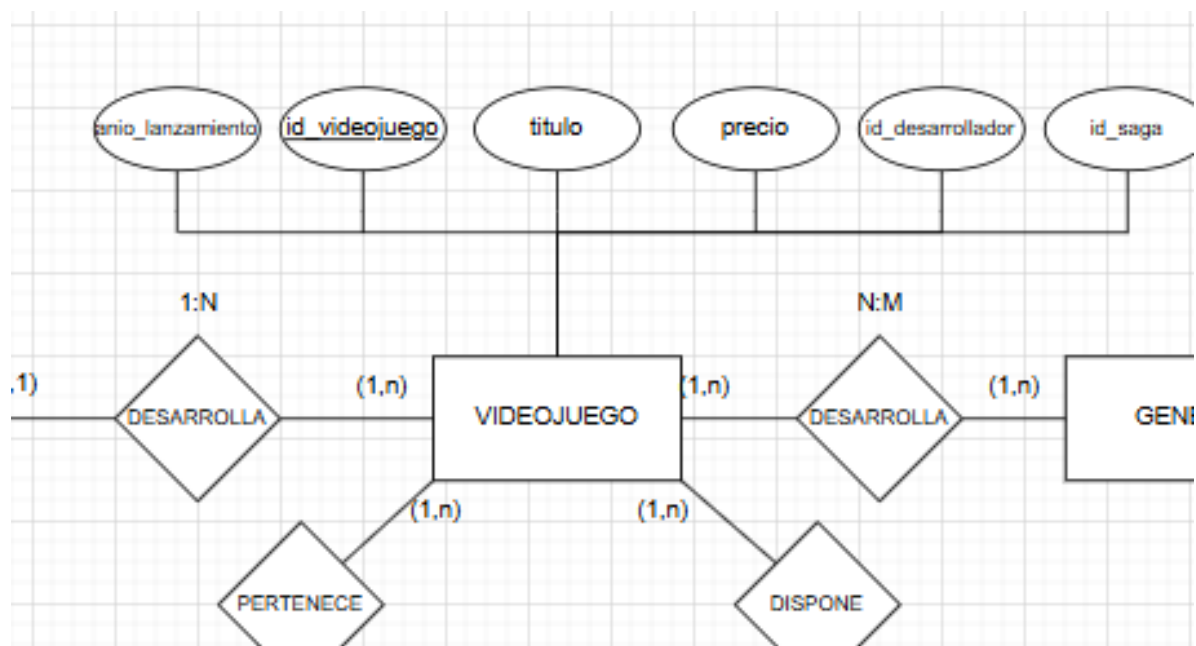
como clave principal.

La siguiente tabla es la plataforma (playstation, xbox, steam, etc.) para cuál disponible el videojuego en nuestro catálogo y para poder identificarlo correctamente, la añadiremos a nuestra base de datos en donde estará con 3 atributos, que se aprecian en la imagen



La entidad genero representa las categorías temáticas o estilísticas a las que puede pertenecer un videojuego (acción, aventura, RPG, estrategia, deportes, etc.) Cada género se identifica de forma única mediante el atributo id_genero, que actúa como clave primaria, y dispone además de un atributo nombre, que describe el tipo de género.

Para resolver esta multiplicidad en el modelo relacional, se crea una tabla intermedia que contiene las claves primarias de ambas entidades (id_videojuego e id_genero). Esta tabla evita duplicidades, mantiene la integridad referencial y permite representar correctamente la relación N:M sin necesidad de incluir atributos redundantes dentro de videojuego.



VIDEOJUEGO es la entidad central del modelo y representa cada juego del catálogo.

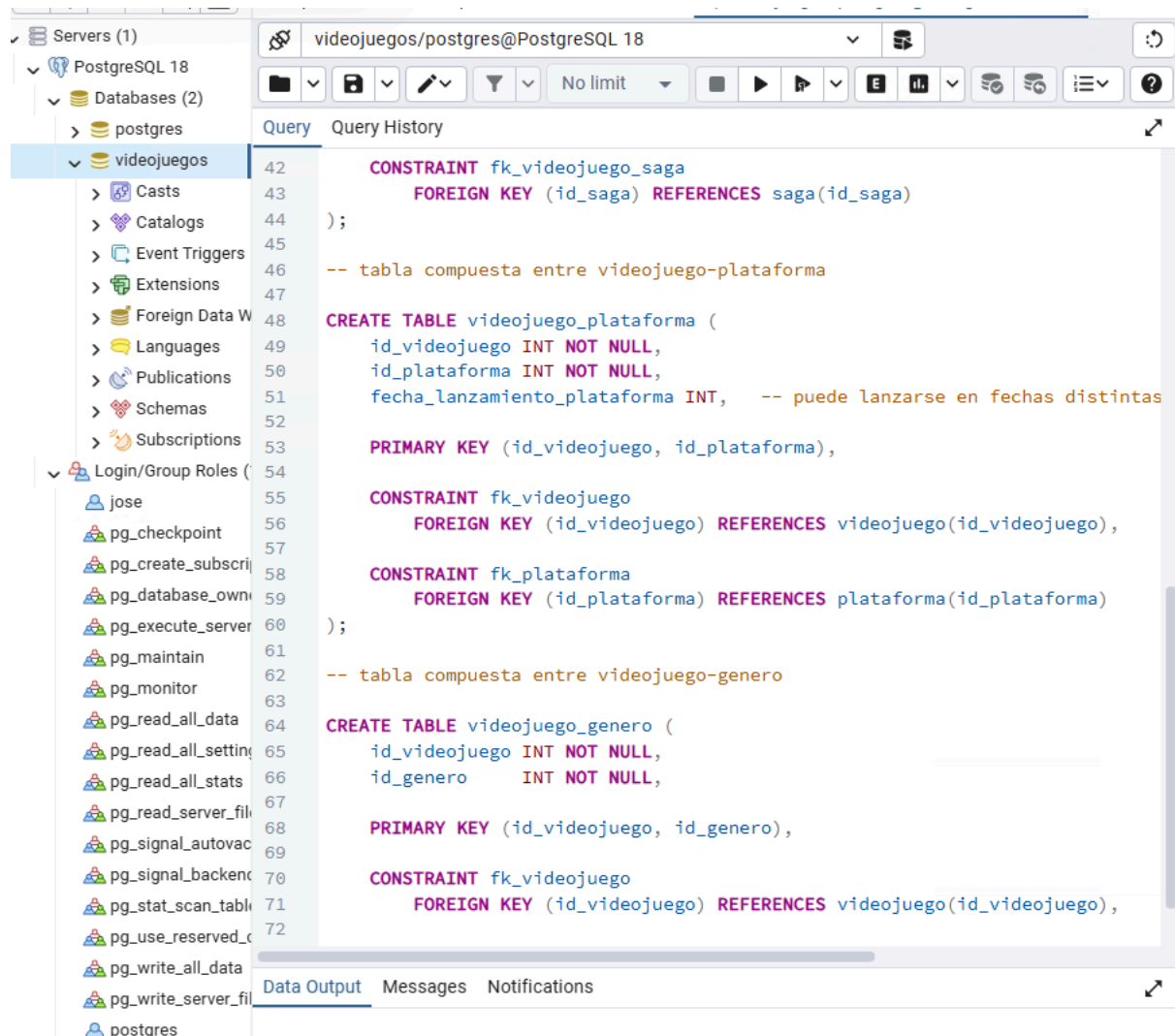
Incluye atributos como id_videojuego (clave primaria), título, año_lanzamiento, precio e id_desarrollador, que permiten identificar el juego, describirlo y vincularlo con su desarrollador. Desde VIDEOJUEGO se establecen relaciones con otras entidades del modelo: Con DESARROLLADOR, mediante una relación 1:N, ya que un desarrollador puede crear varios videojuegos.

Con GÉNERO y PLATAFORMA, mediante relaciones N:M, lo que permite que un videojuego tenga varios géneros y esté disponible en múltiples plataformas.

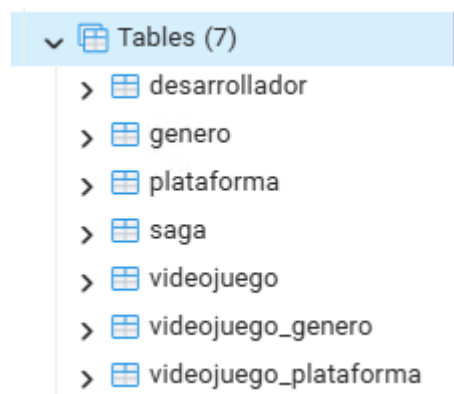
Con SAGA, mediante una relación 0,1 a N, ya que un videojuego puede pertenecer o no a una saga concreta

Consultas sql y exportación a JSON

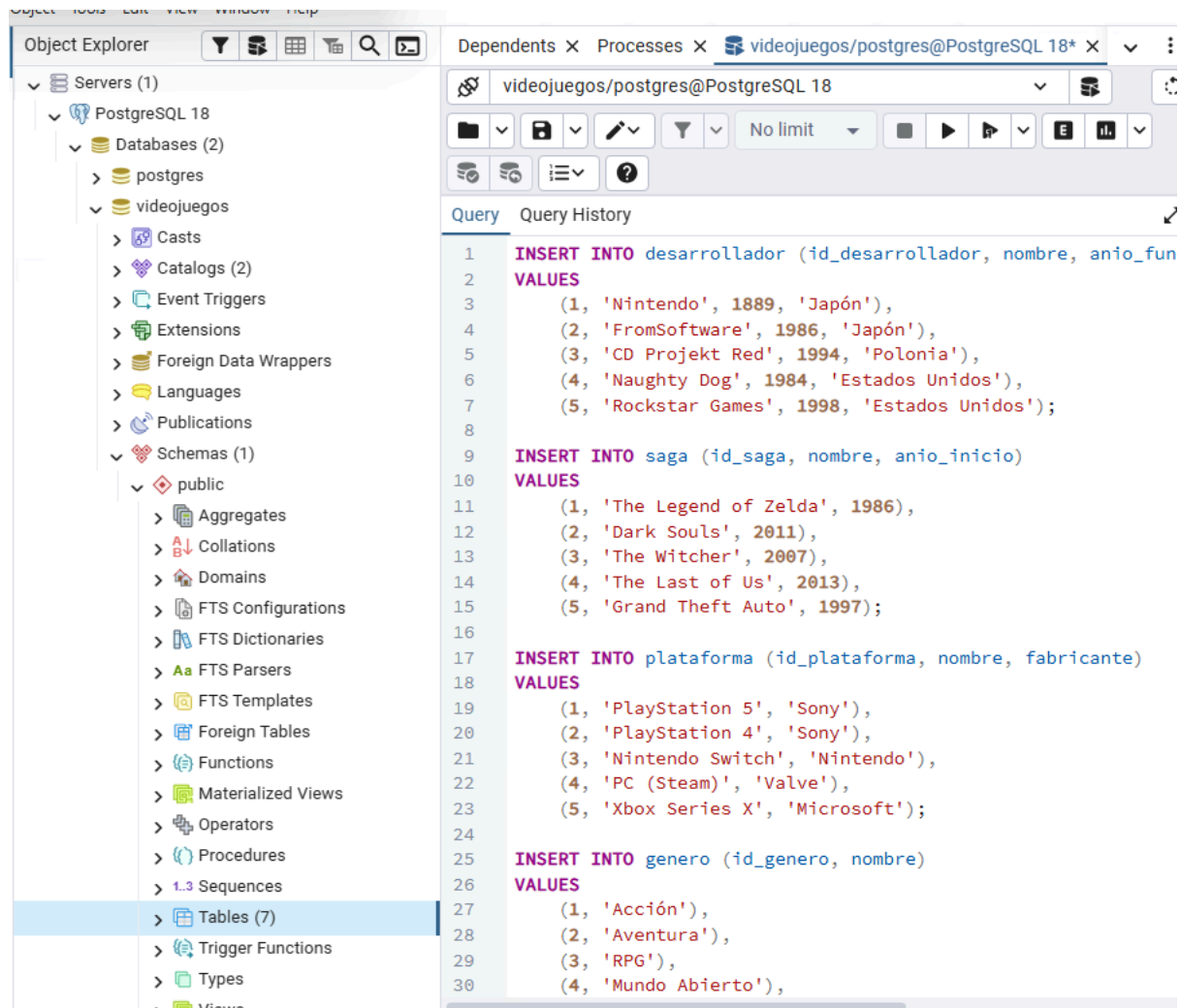
Primero colocamos la estructura con el script ddl en la base de datos la cual ha sido entregada en la carpeta sql



Y aquí podemos observar como las tablas han sido ejecutadas con éxito



Lo siguiente será el script dml, para ello añadiremos datos y posteriormente serán introducidos en la base de datos , los scripts se encuentran en la carpeta de sql



En el añadiremos la información necesaria para que nuestra base de datos tenga cuerpo y podamos trabajar correctamente con ella

Para realizar las consultas y almacenar los datos en archivos en formato JSON utilizaremos la funcion json_agg la cual se encargará en cada consulta agrupar las filas dentro de un array JSON y de esta forma poder leerlo correctamente.

Colocamos el script que se encuentra en la carpeta sql dentro de nuestra base de datos

Esta consulta obtiene todos los videojuegos junto con su desarrollador y, si existe, la saga a la que pertenecen. Se utilizan JOIN y LEFT JOIN para relacionar las tablas y ORDER BY para ordenar por año de lanzamiento.

El resultado se exporta como un array JSON mediante json_agg.

```
SELECT json_agg(resultado) AS datos_json
FROM (
    SELECT
        v.id_videojuego,
        v.titulo,
        v.anio_lanzamiento,
        v.precio,
        d.nombre AS desarrollador,
        s.nombre AS saga
    FROM videojuego v
    JOIN desarrollador d
    ON v.id_desarrollador = d.id_desarrollador
    LEFT JOIN saga s
    ON v.id_saga = s.id_saga
    ORDER BY v.anio_lanzamiento DESC
) resultado;
```

Esta consulta muestra cada videojuego acompañado de todos sus géneros.

Se resuelve la relación N:M mediante la tabla intermedia videojuego_genero. Genera un array JSON con los géneros asociados a cada título.

```
SELECT json_agg(resultado) AS datos_json
FROM (
    SELECT
        v.titulo,
```

```

        v.precio,
        json_agg(json_build_object('genero', g.nombre)) AS generos
FROM videojuego v
JOIN videojuego_genero vg
ON v.id_videojuego = vg.id_videojuego
JOIN genero g
ON vg.id_genero = g.id_genero
GROUP BY v.titulo, v.precio
ORDER BY v.titulo
) resultado;

```

La consulta calcula cuántos videojuegos ha creado cada desarrollador y el precio medio de sus títulos. Se usan funciones de agregación (COUNT, AVG) y un LEFT JOIN para incluir desarrolladores sin juegos.

```

SELECT json_agg(resultado) AS datos_json
FROM (
    SELECT
        d.nombre AS desarrollador,
        d.pais,
        COUNT(v.id_videojuego) AS total_juegos,
        ROUND(AVG(v.precio), 2) AS precio_medio
    FROM desarrollador d
    LEFT JOIN videojuego v ON d.id_desarrollador = v.id_desarrollador
    GROUP BY d.nombre, d.pais
    ORDER BY total_juegos DESC
) resultado;

```

Filtra los videojuegos que están disponibles en una plataforma concreta por ejemplo, PC. Se utiliza la tabla intermedia videojuego_plataforma para resolver la relación N:M. El resultado se exporta como JSON con título, año, precio y plataforma.

```

SELECT json_agg(resultado) AS datos_json
FROM (
    SELECT
        v.titulo,
        v.anio_lanzamiento,
        v.precio,
        p.nombre AS plataforma
    FROM videojuego v
    JOIN videojuego_plataforma vp
    ON v.id_videojuego = vp.id_videojuego
    JOIN plataforma p
    ON vp.id_plataforma = p.id_plataforma
    WHERE p.nombre = 'PC (Steam)'
    ORDER BY v.titulo
) resultado;

```

Agrupar los videojuegos por saga, mostrando número total de juegos, suma de precios y un array con los títulos pertenecientes a cada una.

Incluye una subconsulta con `json_agg` para generar un JSON anidado dentro del JSON principal. Es la consulta más completa, combinando agrupación, agregaciones y estructura jerárquica en JSON.

```

SELECT json_agg(resultado) AS datos_json
FROM (
    SELECT
        s.nombre AS saga,
        s.anio_inicio,
        COUNT(v.id_videojuego) AS total_juegos,
        SUM(v.precio) AS precio_total,
        (
            SELECT json_agg(json_build_object(
                'titulo', v2.titulo,

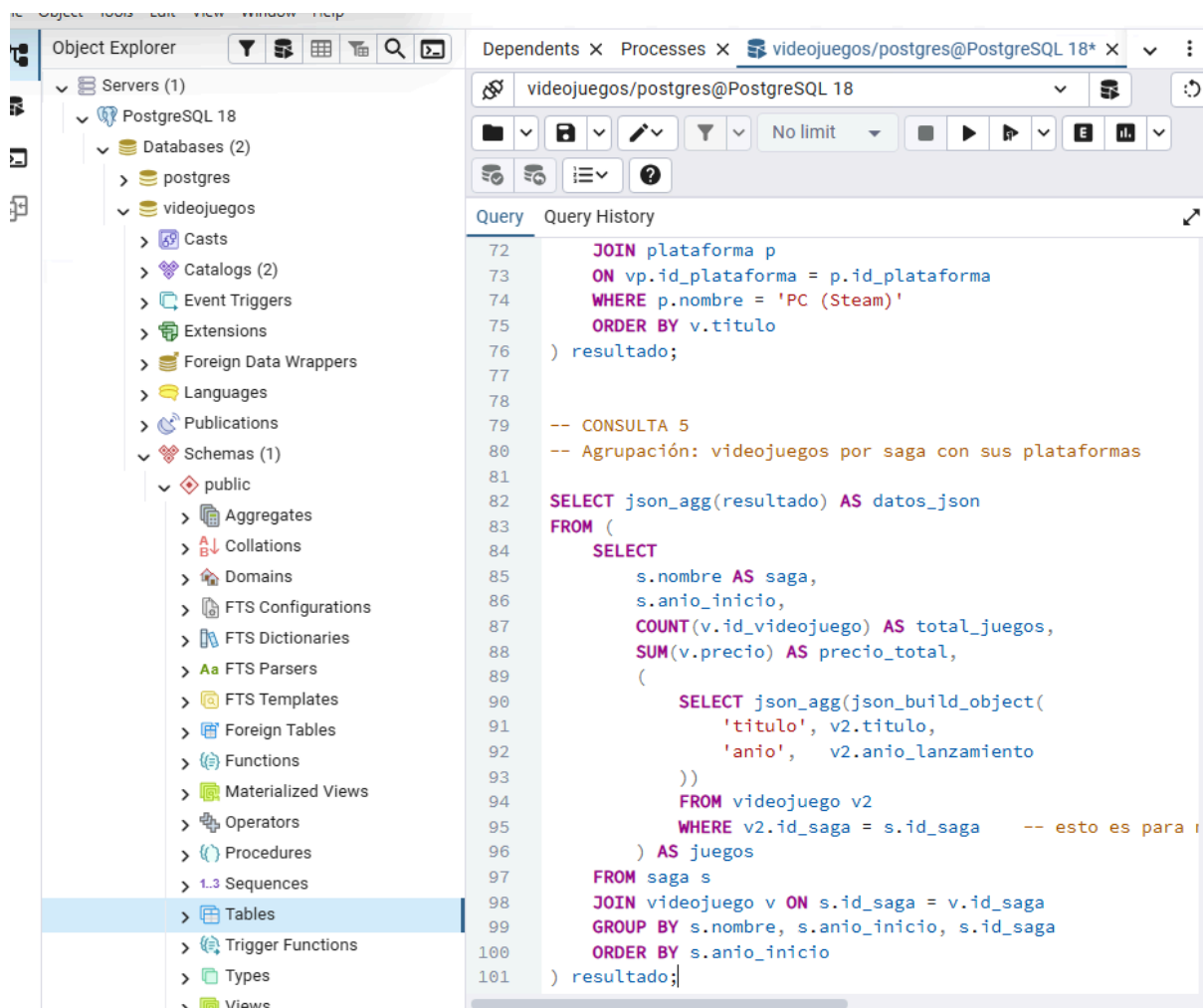
```

```

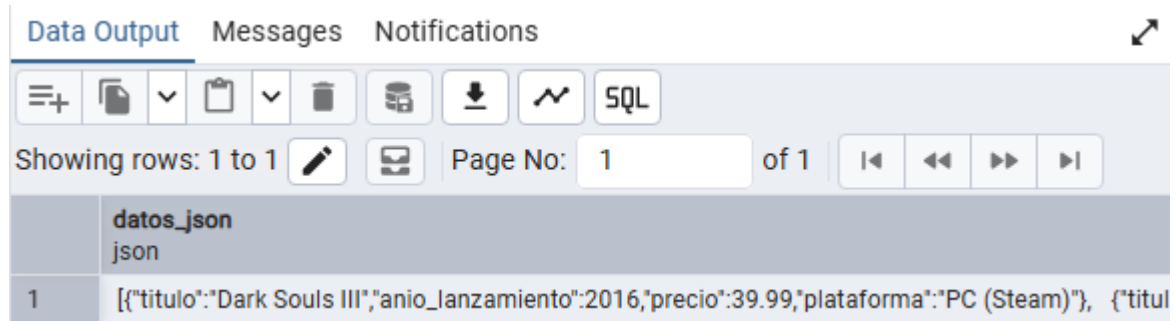
        'anio',    v2.anio_lanzamiento
    ))
    FROM videojuego v2
        WHERE v2.id_saga = s.id_saga    -- esto es para meterlos
dentro de un array propio con sus respectivos datos
    ) AS juegos
FROM saga s
JOIN videojuego v ON s.id_saga = v.id_saga
GROUP BY s.nombre, s.anio_inicio, s.id_saga
ORDER BY s.anio_inicio
) resultado;

```

Colocamos los script dentro de nuestra base de datos y lo ejecutamos



Nos entregará un archivo que contendrá toda la información de la consulta en formato json, lo que haremos será guardarlo para luego utilizarlo



The screenshot shows a database management interface with tabs for 'Data Output', 'Messages', and 'Notifications'. Below the tabs is a toolbar with icons for various actions like adding, deleting, and exporting. A status bar indicates 'Showing rows: 1 to 1' and 'Page No: 1 of 1'. The main area displays a table with one row of JSON data. The first column is labeled 'datos_json' and the second column is labeled 'json'. The JSON data is: [{"titulo": "Dark Souls III", "anio_lanzamiento": 2016, "precio": 39.99, "plataforma": "PC (Steam)"}].

datos_json	json
1	[{"titulo": "Dark Souls III", "anio_lanzamiento": 2016, "precio": 39.99, "plataforma": "PC (Steam)"}, {"titul

```
{ } consulta1.json
{ } consulta2.json
{ } consulta3.json
{ } consulta4.json
{ } consulta5.json
```

Aplicación web con JavaScript básico

La parte web del proyecto tiene como finalidad mostrar de forma clara e interactiva los datos que previamente se han obtenido desde PostgreSQL y exportado en formato JSON. Para ello se ha desarrollado una pequeña aplicación utilizando únicamente HTML, CSS y JavaScript

La idea principal es que el usuario pueda cargar cualquiera de los archivos JSON y visualizar su contenido de manera ordenada, pudiendo además interactuar con los datos mediante varias funciones básicas implementadas con JavaScript

- **index.html** contiene la estructura principal de la interfaz.
- **style.css** define los estilos básicos de la página.
- **script.js** incluye toda la lógica de carga de datos, filtrado, ordenación y detalle.
- La carpeta **json/** almacena los ficheros generados desde PostgreSQL.

La interfaz se ha diseñado de forma sencilla y funcional. En la parte superior se encuentran cinco botones, cada uno asociado a uno de los archivos JSON del proyecto. Los botones están nombrados según el contenido real de cada consulta (Videojuegos, Géneros por juego, Desarrolladores, Juegos en PC y Sagas), lo que facilita al usuario saber qué tipo de información va a cargar, debajo de los botones se incluye una pequeña barra de herramientas con dos funciones:

Un buscador, que permite filtrar por título. Un botón para ordenar por precio, aplicable a las consultas que contienen ese campo.

Catálogo de Videojuegos

Datos cargados desde archivos JSON generados en PostgreSQL

Videojuegos

Géneros por juego

Desarrolladores

Juegos en PC

Sagas

Ordenar por precio

Dark Souls III

Elden Ring

GTA V

The Last of Us Part II

The Witcher 3

Zelda: Breath of the Wild

Zelda: Tears of the Kingdom

The Witcher 3

Precio: 39.99 €

Géneros: RPG, Mundo Abierto

En la parte central se muestra la lista de elementos cargados, y al seleccionar cualquiera de ellos aparece un panel de detalle con información ampliada.

La carga de los datos se realiza mediante la función `fetch()`, que permite leer los archivos JSON de la carpeta correspondiente. Cuando el usuario pulsa uno de los botones, se carga el archivo asociado y se almacena en una variable interna para poder trabajar con él.

```
// Cargar JSON según botón
botones.forEach((boton, index) => {
  boton.addEventListener("click", () => {
    const archivo = boton.getAttribute("data-json");
    tipoConsulta = index + 1;

    fetch("../json/" + archivo)
      .then(res => res.json())
      .then(datos => {
        datosCargados = datos;
        mostrarDatos(datosCargados);
      });
  });
});
```

Una vez cargados, los datos se muestran en pantalla generando dinámicamente los elementos mediante JavaScript.

La práctica exige implementar al menos tres interacciones con el usuario. En esta aplicación se han desarrollado las siguientes:

Filtro por título

El usuario puede escribir en el buscador y la lista se actualiza automáticamente mostrando solo los elementos cuyo título coincida con el texto introducido.

Elden Ring

GTA V

Zelda: Tears of the Kingdom

GTA V

Ordenación por precio

Permite ordenar los elementos de menor a mayor precio en aquellas consultas que incluyen este campo.

Buscar por título...	Ordenar por precio
---	---

GTA V
Dark Souls III
The Witcher 3
The Last of Us Part II
Elden Ring
Zelda: Breath of the Wild
Zelda: Tears of the Kingdom

Visualización de detalle

Al hacer clic sobre cualquier elemento de la lista, se muestra una ficha con información ampliada adaptada al tipo de JSON cargado.

GTA V

Precio: 29.99 €

Géneros: Acción, Mundo Abierto

Estas tres funciones cumplen los requisitos de interacción y demuestran el uso de arrays, eventos y manipulación del DOM.

El diseño de la página se ha mantenido sencillo, utilizando únicamente CSS básico. Se han aplicado estilos mínimos para mejorar la legibilidad.

CONCLUSIÓN

En esta parte del proyecto hemos desarrollado una aplicación web sencilla pero totalmente funcional, cuyo objetivo principal es mostrar los datos generados desde PostgreSQL en formato JSON y permitir al usuario interactuar con ellos de forma clara. Para ello hemos utilizado únicamente HTML, CSS y JavaScript, tal y como se pedía en la práctica, sin frameworks ni librerías externas.

La web carga cada uno de los cinco archivos JSON mediante `fetch()` y adapta la visualización según la estructura de cada consulta, ya que cada JSON contiene campos distintos. Esto nos obligó a preparar un script capaz de detectar qué tipo de datos se está mostrando y ajustar tanto la lista como el panel de detalle para evitar errores y mostrar siempre información coherente. Gracias a esto, la aplicación funciona correctamente con todos los JSON generados en la parte de SQL.

Además, hemos implementado las tres interacciones mínimas que se exigían, un buscador para filtrar por título, un botón para ordenar por precio y un panel de detalle que aparece al hacer clic en cualquier elemento. Estas funciones demuestran el uso de eventos, arrays y manipulación del DOM, que era uno de los puntos importantes del trabajo.

WEBGRAFÍA:

<https://vandal.elespanol.com/rankings/videojuegos>

<https://es.ign.com/portal-2/178635/feature/los-100-mejores-videojuegos-de-la-historia>

https://neon.com/docs/functions/json_agg